

ⓘ Shared by NitiVerse. This is a copy of a chat between Claude and NitiVerse. Content may include unverified or unsafe content that does not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

Files hidden  
when shared



<https://github.com/arundhatikadlabal/SE-Labs-PES1UG24CS083.git>

[https://github.com/SETAPESU26/19\\_battleship.git](https://github.com/SETAPESU26/19_battleship.git)

this is my personal repo, and the second one is the one i need to work with  
read the docs and help me follow the instructions step by step

I'll clone the assigned battleship repo and read its README to see what's broken and which features you need to add.

Ran a command

The repo is cloned. Next I'll read the source files to find the coordinate bug before changing anything, as the README advises.

Read 5 files

I've read all the code. Before explaining the bug, I'll reproduce it, since the checklist requires showing the original defect.

Ran a command

I've cloned `19_battleship`, read all five files, and reproduced the bug. Nothing is changed or pushed yet.

### What's broken (Task 1 evidence)

I played a scripted game by typing `3,3`, `3,4`, `3,5`. The player's shots hit correctly, but three things are inconsistent:

1. **The AI reports false hits.** `ai.py` returns a 1-indexed string like `"1,2"`. `game.py` parses it as `(1, 2)` without subtracting 1, so the two modules disagree about coordinates. In my run the AI "hit" `1,2` and `1,1`, but your ships actually occupy `(1,1)`, `(1,2)` and `(1,3)` in 0-indexed terms, which are the cells displayed as `2,2`, `2,3` and `2,4`.
2. **The AI never fires on your board.** `self.player.fire()` is never called, so your board's `shots` stay empty and you can never lose.
3. **The "Ship cells remaining" counter never goes down.** `show()` compares `enemy.ships` against `player.shots`, but your shots are stored in `enemy.shots`. It stayed at 3 after a hit.

The fix for Task 1 is to use 0-indexed `(row, col)` tuples internally everywhere, and convert to 1-indexed only when printing or reading input.

### Your step-by-step plan

1. **Done:** clone the assigned repo and read the README and code.
2. **Yours to do:** record a 10-second "before" video of `python main.py`. Fire at `3,3`, then watch the counter stay at 3 and the AI report hits on empty cells.

3. **Task 1:** consistent coordinates. Commit 1.
4. **Task 2:** multiple ships, per-ship damage, sunk detection, no repeated shots, game over when the fleet is sunk. Commit 2.
5. **Task 3:** AI targets neighbours after a hit, never repeats, and handles a full board. Commit 3.
6. **Task 4:** hit, miss and sunk messages appear exactly once per real shot. Commit 4.
7. **Testing:** cover hits, misses, repeated shots, sinking one and all ships, AI repeats, adjacent targeting, invalid input and quitting.
8. **Yours to do:** record the 10-second "after" video.
9. **Push:** copy the code into a `Lab-4/` folder in your own repo ( `SE-Labs-PES1UG24CS083` ), with the videos and exported chat. Do not open a PR to `SETAPESU26`.
10. **Submit:** use the Google Form with your repo link, the PDF, the video links, and the prompts or chat link.

I can't record the videos or export this chat history for you. I also can't push to your repo without your GitHub credentials, so I'd prepare the commits locally and you'd run the final push.

Do you want me to go ahead and implement Tasks 1-4 now, one commit each? If so, record your "before" video first, because it needs the original broken code.

nono ill do everything - just tell me how to do it step by step

Here's the whole lab in order. Every command is something you run yourself.

### Step 1: Set up your folders

```
bash
mkdir lab4 && cd lab4
git clone https://github.com/SETAPESU26/19_battleship.git
git clone https://github.com/arundhatikadlabal/SE-Labs-PES1UG24CS083.git
cd 19_battleship
python main.py          # needs Python 3.9+; type q to quit
```

Don't push anything to `19_battleship`. It's read-only for you, and you must not raise a PR to it.

### Step 2: Record the "before" video (10 seconds)

Use OBS, Win+G (Xbox Game Bar) on Windows, or QuickTime/Cmd+Shift+5 on Mac. Run the game and type:

1. `3,3`, which is a real enemy ship cell. It says HIT, but "Ship cells remaining" stays at 3.
2. Another shot or two. The AI says "AI scored a hit" on cells that are empty on your board.

Save it as `before.mp4`. This is your evidence of the original bug, so record it before you edit anything.

### Step 3: Set up your personal repo

```
bash
```

```
cd ../SE-Labs-PES1UG24CS083
mkdir -p Lab-4
cp ../19_battleship/*.py ../19_battleship/README.md ../19_battleship/requirements.tx
git add Lab-4
git commit -m "Lab 4: add original battleship code (baseline)"
```

This copies only the files, not the `.git` folder. Do all edits from now on inside `SE-Labs-PES1UG24CS083/Lab-4/`.

## Step 4: Fix the code, one task per commit

Open your LLM and paste the relevant files first. Ask it to explain before it changes anything, as the README requires. After each task, run the game, test it, then commit.

### Task 1 — Coordinates

Prompt: *"Here is my board.py, game.py and ai.py. Explain how coordinates are represented in each module. Then make them consistent: use 0-indexed (row, col) tuples internally everywhere, and convert to 1-indexed only when reading input or printing. Also make the AI actually fire on the player's board. Keep the changes minimal."*

Check that your shots hit exactly on ship cells and the AI never reports a hit on an empty cell.

```
bash
git commit -am "Task 1: consistent coordinate handling"
```

### Task 2 — Fleet and win logic

Prompt: *"Support multiple ships per board, track damage per ship, announce when a specific ship is sunk, reject repeated shots, and end the game when the whole fleet is sunk. Also end it if the AI sinks the player's fleet. Keep state in memory only and don't add dependencies."*

```
bash
git commit -am "Task 2: multi-ship fleet and win logic"
```

### Task 3 — Smarter AI

Prompt: *"After an AI hit, make it prefer untried cells adjacent (up/down/left/right) to the hit. It must never shoot the same cell twice and must handle having no cells left without crashing."*

```
bash
git commit -am "Task 3: AI targets neighbours after a hit"
```

### Task 4 — Feedback

Prompt: *"Make sure hit/miss/sunk messages print exactly once per real shot, and that the AI's internal candidate selection never prints hit/miss messages."*

```
bash
git commit -am "Task 4: shot-level feedback"
```

If a prompt doesn't work, refine it, but try to get each task in 3-4 prompts as the handout says.

## Step 5: Test everything from the README

Play through or script each of these, using `printf '3,3\nq\n' | python main.py` for quick checks:

- a hit and a miss
- the same shot twice (should say "Already fired there")
- sinking one ship, then sinking all ships (game ends)
- the AI never repeating a coordinate, and targeting adjacent cells after a hit
- invalid input such as `abc`, `0,0`, `7,7` and `1,1,1`
- `q` to quit

## Step 6: Record the "after" video (10 seconds)

Show a hit, a miss, a "ship sunk" message, an AI shot, and a repeated-shot rejection. Save it as `after.mp4`.

## Step 7: Export your chat history

Share your LLM conversation. In ChatGPT, Claude or Gemini, use the share-link feature. Also export the chat as PDF, or copy the prompts and screenshots into a document, since the handout asks for a doc/PDF. Make sure the link is accessible to anyone who has it.

## Step 8: Push to your repo

Put `before.mp4`, `after.mp4` and the chat PDF into `Lab-4/`. If the videos are large, upload them to Google Drive or YouTube (unlisted) instead and use the links.

```
bash

git add Lab-4
git commit -m "Lab 4: add videos and chat history"
git push origin main
git log --oneline      # confirm you see a separate commit per task
```

Check on GitHub that `Lab-4/` has the updated code and files.

## Step 9: Submit the Google Form

Open <https://forms.gle/qRvtXsBXf3cr7PAN6> and submit:

1. Your repo link
2. The PDF with evidence of all tasks
3. The before and after video links (set sharing to "anyone with the link")
4. The prompts or chat link

Your email's deadline line is cut off ("The submission deadline i..."), so check the original email or ask the instructor for the date.

If you get stuck on any step, paste the error or the code here and I'll help you debug it.

okay wait lets do one step at a time, until im done with the step and say go on -  
dont go to the next  
now for step1- where do i type these commands?

You type them in a **terminal** on your own computer, not in this chat.

## Opening a terminal

- **Windows:** press the Windows key, type `cmd` or `PowerShell`, and press Enter. If you've installed Git for Windows, "Git Bash" also works and is the closest match to the commands I gave.
- **Mac:** press Cmd+Space, type `Terminal`, and press Enter.
- **Linux:** press Ctrl+Alt+T.
- **VS Code (if you use it):** go to Terminal → New Terminal. This is handy because you'll edit code there later.

Before the commands, check that git and Python are installed. Type these one at a time:

```
bash

git --version
python --version
```

On Mac or Linux, use `python3 --version` if `python` isn't found. If either says "not found" or "not recognized", install it first: git from <https://git-scm.com> and Python 3.9+ from <https://python.org>. On Windows, tick "Add Python to PATH" in the installer.

**Where the folder will go.** The terminal opens in your home folder, which is fine. `mkdir lab4` creates the new folder there. If you'd rather keep it on your Desktop, first type `cd Desktop`.

**Windows note.** `mkdir lab4 && cd lab4` works in PowerShell 7 and Git Bash. If it complains in older PowerShell or cmd, run `mkdir lab4`, then `cd lab4` as two separate lines.

Then run the Step 1 commands, one block at a time. When you're done, or if you hit an error, tell me what you see and I'll move on only when you say go on.

```
PS C:\Users\ARUNDHATI> git --version
git version 2.52.0.windows.1
PS C:\Users\ARUNDHATI> python --version
Python 3.10.0
PS C:\Users\ARUNDHATI>
```

Both are installed, and Python 3.10 meets the 3.9+ requirement. You're in PowerShell, so use these Step 1 commands. I've split `mkdir lab4 && cd lab4` into separate lines, since older PowerShell versions don't accept `&&`:

```
powershell

mkdir lab4
cd lab4
git clone https://github.com/SETAPESU26/19_battleship.git
git clone https://github.com/arundhatikadlabal/SE-Labs-PES1UG24CS083.git
cd 19_battleship
python main.py
```

Here is what to expect:

- After the two `git clone` lines, you should see "Cloning into..." for each.
- `python main.py` starts the game and prints `Battleship`, followed by a `>` prompt.
- Type `q` and press Enter to quit the game.

If the second clone asks you to sign in to GitHub, log in through the window that pops up.

Tell me what you see, or paste any error, and I'll wait for your "go on" before Step 2.

```
PS C:\Users\ARUNDHATI> cd lab4
PS C:\Users\ARUNDHATI\lab4> git clone
https://github.com/SETAPESU26/19\_battleship.git
Cloning into '19_battleship'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 11 (delta 0), reused 0 (delta 0), pack-reused 8 (from 1)
Receiving objects: 100% (11/11), done.
Resolving deltas: 100% (2/2), done.
PS C:\Users\ARUNDHATI\lab4> git clone https://github.com/arundhatikadlabal/SE-Labs-PES1UG24CS083.git
Cloning into 'SE-Labs-PES1UG24CS083'...
remote: Enumerating objects: 92, done.
remote: Counting objects: 100% (92/92), done.
remote: Compressing objects: 100% (84/84), done.
remote: Total 92 (delta 29), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (92/92), 2.26 MiB | 8.86 MiB/s, done.
Resolving deltas: 100% (29/29), done.
PS C:\Users\ARUNDHATI\lab4> cd 19_battleship
PS C:\Users\ARUNDHATI\lab4\19_battleship> python main.py
Battleship

Your shots are coordinates like 2,3.
Ship cells remaining: 3
>
```

Step 1 is done. Both repos are cloned and the game runs. It's waiting at the `>` prompt.

Type `q` and press Enter to quit it for now. We'll run it again when you record the "before" video in Step 2.

Say "go on" when you're ready for Step 2.

go on

## Step 2: Record the "before" video (10 seconds)

You need a 10-second screen recording of the original, unchanged game showing the bug.

**1. Pick a recorder.** On Windows the easiest is the built-in Xbox Game Bar:

- Click on your terminal window so it's the active one.
- Press **Win + G** to open Game Bar.
- Click the record button (the circle) in the Capture panel, or press **Win + Alt + R** to start and stop.
- If Game Bar doesn't work, use Snipping Tool (Win + Shift + S, then switch to the video option) or OBS Studio.

Recordings from Game Bar are saved to `C:\Users\ARUNDHATI\Videos\Captures`.

**2. Get ready before you hit record.** In the terminal, run:

```
powershell
```

```
cd C:\Users\ARUNDHATI\lab4\19_battleship
python main.py
```

Make the terminal window large enough to read, with the `>` prompt visible.

### 3. Start recording, then type these, one at a time:

1. `3,3` and press Enter. It says HIT, but "Ship cells remaining" still shows 3, and the AI says "AI scored a hit" even though your ships weren't really hit.
2. `3,4` and press Enter. The same thing happens again.
3. `q` to quit.

Pause about a second after each output so the video is readable, then stop recording at around 10 seconds.

**4. Rename the file** to `before.mp4` and move it somewhere easy to find, such as your `lab4` folder. Don't put it inside `19_battleship`.

Play the video back once to check that the hit message, the unchanged counter and the AI line are all clearly visible.

Say "go on" when it's recorded and saved.

can i do it again? or is it one and done?

Yes, you can redo it as many times as you like. Nothing is submitted until Step 9, so retake it until it looks right.

- **Keep only the best take.** Delete or rename the others so you don't get confused about which one is `before.mp4`.
- **The game is random on the AI's side.** Its shots differ every run, so each take will show slightly different AI messages. That's fine, as long as the video shows your hit with the counter staying at 3.
- **Redo as long as the code is unchanged.** The "before" video must show the original broken game, so retake it before you start editing code in Step 4. If you've already modified code, the clean original is still in `C:\Users\ARUNDHATI\lab4\19_battleship`, since we only edit the copy in your personal repo.

Say "go on" when you have a take you're happy with.

```
PS C:\Users\ARUNDHATI\lab4\19_battleship> python main.py
Battleship

Your shots are coordinates like 2,3.
Ship cells remaining: 3
| 3,3
HIT!
AI fired at 3,3

Your shots are coordinates like 2,3.
Ship cells remaining: 3
| 3,4
HIT!
AI fired at 1,1
```

```
AI scored a hit.  
Your shots are coordinates like 2,3.  
Ship cells remaining: 3  
| q  
PS C:\Users\ARUNDHATI\lab4\19_battleship>  
Show more
```

Your output shows the bug clearly:

- Both of your shots said HIT, but "Ship cells remaining" stayed at 3 each time.
- The AI said "AI fired at 1,1" and "AI scored a hit", but your ships are really at cells 2,2, 2,3 and 2,4 in the numbers the game displays, so 1,1 should have been a miss.

The AI's first shot, 3,3, had no hit message, which is also correct. It's the second shot that shows the false hit.

Those two things are what the video should capture. Make sure your recording includes the counter staying at 3 and the AI's "scored a hit" line, then save the file as `before.mp4`.

Say "go on" once the video is saved and I'll give you Step 3.

go on

### Step 3: Copy the original code into your personal repo

You'll work from your personal repo from now on, so the code goes into a `Lab-4` folder there.

#### 1. Go into your repo and look around:

```
powershell  
cd C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083  
dir
```

Check what's in there. If you see folders for earlier labs, such as `Lab-1` or `Lab-3`, note how they're named. If a `Lab-4` folder already exists, tell me before continuing.

#### 2. Create the folder and copy only the project files:

```
powershell  
mkdir Lab-4  
Copy-Item ..\19_battleship\main.py, ..\19_battleship\game.py, ..\19_battleship\board  
dir Lab-4
```

The last command should list exactly 6 files. We're deliberately not copying the hidden `.git` folder from the assigned repo.

#### 3. Make the baseline commit:

```
powershell
```

```
git add Lab-4
git commit -m "Lab 4: add original battleship code (baseline)"
git status
```

`git status` should say your working tree is clean.

If the commit fails with "Please tell me who you are", set your identity once and then rerun the commit:

```
powershell

git config --global user.name "Your Name"
git config --global user.email "your-github-email@example.com"
```

Don't run `git push` yet. We'll push at the end.

Paste the output of `dir Lab-4` and `git status` here, and say "go on" when it all looks right.

```
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083> Copy-Item
..\19_battleship\main.py, ..\19_battleship\game.py, ..\19_battleship\board.py,
..\19_battleship\ai.py, ..\19_battleship\README.md,
..\19_battleship\requirements.txt -Destination Lab-4
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083> dir Lab-4

Directory: C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4

Mode                LastWriteTime         Length Name
----                -
-a----       07-10-2026   20:37           421 ai.py
-a----       07-10-2026   20:37           340 board.py
-a----       07-10-2026   20:37          1805 game.py
-a----       07-10-2026   20:37            83 main.py
-a----       07-10-2026   20:37          3137 README.md
-a----       07-10-2026   20:37           112 requirements.txt

PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083> git add Lab-4
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083> git commit -m "Lab 4:
add original battleship code (baseline)"
[main 5d8385b] Lab 4: add original battleship code (baseline)
6 files changed, 185 insertions(+)
create mode 100644 Lab-4/README.md
create mode 100644 Lab-4/ai.py
create mode 100644 Lab-4/board.py
create mode 100644 Lab-4/game.py
create mode 100644 Lab-4/main.py
create mode 100644 Lab-4/requirements.txt
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083> git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083>
```

Step 3 is done. All 6 files are in `Lab-4`, and the baseline commit (`5d8385b`) is saved on `main`. The "ahead of 'origin/main' by 1 commit" line is expected, because nothing has been pushed yet.

Say "go on" when you're ready for Step 4, where you start fixing the code with your LLM.

## Step 4: Fix the code with an LLM (one task at a time)

Step 4 has four tasks, and each gets its own commit, so we'll do them one by one. This is **Task 1: consistent coordinates**.

**1. Start a fresh chat in the LLM you'll use for the whole lab** (ChatGPT, Gemini, Claude and so on). Use this one chat for all four tasks, because the complete history is a deliverable. Don't use this conversation for the lab prompts.

**2. Show the LLM your code.** In PowerShell, print the files and copy the output:

```
powershell

cd C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4
type board.py
type game.py
type ai.py
```

You can also open the files in VS Code or Notepad and copy the text.

**3. Send this as your first prompt, with the three files pasted below it:**

Here is my Python terminal Battleship game (board.py, game.py, ai.py). Before changing anything, explain how coordinates are represented in each module and where they disagree. Then fix it so coordinates are consistent: use 0-indexed (row, col) tuples internally everywhere, and convert to 1-indexed only when reading user input or printing. Also make the AI's shots actually land on the player's board, and fix the "Ship cells remaining" counter. Keep the changes minimal and don't replace the project. Show the full updated files.

Read the explanation. The README says you're responsible for understanding what changes. If something is unclear, ask the LLM about it.

**4. Apply the changes.** Replace the contents of `board.py`, `game.py` and `ai.py` in `Lab-4` with the LLM's versions, if it changed them.

**5. Test it:**

```
powershell

python main.py
```

Check these:

- `3,3`, `3,4` and `3,5` are all HIT, and the counter goes down each time.
- `1,1` is a MISS.
- The AI only says "scored a hit" when it fires at one of your actual ship cells, which are displayed as `2,2`, `2,3` and `2,4`.

If something is wrong, tell the LLM exactly what you saw. It should take 3-4 prompts at most.

**6. Commit once it works:**

```
powershell
```

```
git add Lab-4
git commit -m "Task 1: consistent coordinate handling"
git status
```

Paste the output of your test run and `git status` here. Say "go on" when Task 1 is committed and I'll give you Task 2.

is it fine that instead of copy pasting from terminal, i just send terminal screenshots?

Yes, it works, as long as your LLM accepts images (ChatGPT, Gemini and Claude all do). Pasting text is better for the code files, though, for a few reasons:

- **Accuracy:** the LLM can misread characters in a screenshot, such as `1` vs `l` or a missing bracket, and then base its fix on code that isn't yours.
- **Long files:** `game.py` may scroll past one screen, so a screenshot can cut off lines.
- **Your chat export:** text prompts show up cleanly in the PDF/doc you submit, while screenshots can export poorly. Evidence of the prompts is a deliverable.

**The quickest way to copy the code as text** is to have PowerShell put a file on your clipboard, then paste with Ctrl+V:

```
powershell
cd C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4
Get-Content board.py | Set-Clipboard
```

Paste, then repeat with `game.py` and `ai.py`. You can also open the file in VS Code or Notepad, press Ctrl+A then Ctrl+C.

Screenshots are fine for things like error messages or your game's output when you report a problem to the LLM. For the code files, stick with text.

Carry on with the Task 1 steps from before, and say "go on" when Task 1 is committed.

okay then should i just copy terminal output , and just paste it into prompt? thats also fine right?

Yes, that's fine. Pasting terminal output as text works well, both for the code from `type board.py` and for your game's output when you report a problem.

- **Copying from PowerShell:** select the text with your mouse and press Enter or right-click to copy it (in older consoles), or Ctrl+C while text is selected (in Windows Terminal). If it's fiddly, the `Get-Content board.py | Set-Clipboard` trick is easier and copies the whole file cleanly.
- **Paste each file with its name above it,** like `board.py:` followed by the code, so the LLM knows which file is which.
- **Check you got everything.** If a file is long, make sure the top of it wasn't scrolled out of the terminal before you copied.
- **Leading prompt lines don't matter.** A stray `PS C:\...>` line won't hurt, but you can leave it out.

Go ahead with Task 1 now, and say "go on" when it's committed.

okay it gave me 3 new file for this, now how do i change the files quickly?

The quickest way on Windows is to open each file, replace everything in it, and save. Do this for each of the three files ( `board.py` , `game.py` , `ai.py` ):

**1. Go to the folder:**

```
powershell  
cd C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4
```

**2. Open a file.** Use VS Code if you have it, otherwise Notepad:

```
powershell  
code board.py
```

or

```
powershell  
notepad board.py
```

**3. Replace the contents.** Click inside the editor, press Ctrl+A to select everything, then Ctrl+V to paste the LLM's new version of that file over it, and press Ctrl+S to save.

**4. Repeat** for `game.py` and `ai.py` , matching each new version to the right file name.

A few things to watch:

- **Don't create new files.** Because you opened the existing ones, the names stay correct. If Notepad ever asks to create a file, you're in the wrong folder.
- **Copy only the code.** Don't paste the LLM's text around the code block, such as "Here is the updated file:" or the triple backticks, or Python will give a syntax error.
- **You can check what changed** before testing:

```
powershell  
git diff --stat
```

This should list `board.py` , `game.py` and `ai.py` as changed.

Then run `python main.py` and test as in the Task 1 checks. Paste any errors or odd behaviour here if you want help, and say "go on" when Task 1 is committed.

now the testing, should i record it again?

No, don't record anything now. The only videos are the "before" one you already made and a single "after" video at the very end, once all four tasks are done. The after video should show

the finished game, with sunk ships, repeated-shot rejection and the smarter AI, not just the Task 1 fix.

For now, just test Task 1 by running the game and checking:

```
powershell
python main.py
```

1. Type `3,3`, `3,4` and `3,5`. All should say HIT, and "Ship cells remaining" should go from 3 down to 2, 1, then 0.
2. Type `1,1`. It should say MISS.
3. Watch the AI's shots. It should only say "scored a hit" when it fires at one of your real ship cells, which are displayed as `2,2`, `2,3` and `2,4`. Run it a few times, since the AI picks randomly.
4. Type `q` to quit.

If all of that behaves correctly, commit:

```
powershell
git add Lab-4
git commit -m "Task 1: consistent coordinate handling"
git status
```

If something is off, tell me what you typed and what you saw. Say "go on" when Task 1 is committed.

Your shots are coordinates like 2,3.  
Ship cells remaining: 3

| 3,3

HIT!  
AI fired at 4,6

Your shots are coordinates like 2,3.  
Ship cells remaining: 2

| 3,4

HIT!  
AI fired at 4,5

Your shots are coordinates like 2,3.  
Ship cells remaining: 1

| 3,5

HIT!  
You sank the fleet.

PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> 1,1

1

1

PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> python main.py  
Battleship

Your shots are coordinates like 2,3.  
Ship cells remaining: 3

| 1,1

MISS!  
AI fired at 6,2

Your shots are coordinates like 2,3.  
Ship cells remaining: 3

```

| 2,2
MISS!
AI fired at 5,2

Your shots are coordinates like 2,3.
Ship cells remaining: 3

| 2,3
MISS!
AI fired at 5,4

Your shots are coordinates like 2,3.
Ship cells remaining: 3

| 2,4
MISS!
AI fired at 3,1

Your shots are coordinates like 2,3.
Ship cells remaining: 3

| ?
Show more

```

Most of Task 1 looks right:

- **The counter works.** It goes  $3 \rightarrow 2 \rightarrow 1$ , then "You sank the fleet" when it reaches 0. Before, it never moved.
- **Hits and misses are correct.** 3,3, 3,4 and 3,5 are hits, and 1,1, 2,2, 2,3 and 2,4 are misses.
- **The 1,1 / 1 / 1 lines in the middle were just PowerShell.** You typed 1,1 into the shell after the game had ended, so it echoed it back. It's harmless.

**One check is left.** In both runs the AI only fired at cells that are empty on your board (4,6, 4,5, 6,2, 5,2, 5,4, 3,1), so we haven't yet seen it correctly score a hit. Your ships are at 2,2, 2,3 and 2,4.

To test that, run `python main.py` again and keep firing at cells that are **not** 3,3, 3,4 or 3,5. Otherwise you'll sink the fleet and end the game early. For example:

```
1,1 1,2 1,3 1,4 1,5 1,6 2,1 2,5 2,6 4,1 4,2 4,3
```

Keep going until the AI fires at one of 2,2, 2,3 or 2,4. Confirm that:

- it prints "AI scored a hit" on those cells, and
- it does **not** print it for any other cell.

The AI is random, so it may take a dozen or so turns. Type `q` to quit when you've seen it.

If that works, commit Task 1:

```
powershell

git add Lab-4
git commit -m "Task 1: consistent coordinate handling"
git status

```

If the AI fires at one of your ship cells and **doesn't** say it scored a hit, tell me the exact lines you saw. Otherwise, say "go on" once it's committed.

Your shots are coordinates like 2,3.  
Ship cells remaining: 3

| 1,1

MISS!  
AI fired at 1,2

Your shots are coordinates like 2,3.  
Ship cells remaining: 3

| 1,2

MISS!  
Show more  
AI fired at 2,2  
AI scored a hit.

Your shots are coordinates like 2,3.  
Ship cells remaining: 3

| 1,3

MISS!  
AI fired at 5,5

Your shots are coordinates like 2,3.  
Ship cells remaining: 3

| 1,4

MISS!  
AI fired at 2,5

Your shots are coordinates like 2,3.  
Ship cells remaining: 3

| 1,5

MISS!  
AI fired at 2,4  
AI scored a hit.

Your shots are coordinates like 2,3.  
Ship cells remaining: 3

| 1,6

MISS!  
AI fired at 1,3

Your shots are coordinates like 2,3.  
Ship cells remaining: 3

>  
it fired at 2,4

That confirms Task 1. Every AI shot is handled correctly:

| AI fired at | Real ship cell? | Message      |
|-------------|-----------------|--------------|
| 1,2         | no              | none         |
| 2,2         | yes             | scored a hit |
| 5,5         | no              | none         |
| 2,5         | no              | none         |
| 2,4         | yes             | scored a hit |
| 1,3         | no              | none         |

I'm reading your note "it fired at 2,4" as the hit in turn 5 of your output above. If you actually saw the AI fire at 2,4 a second time in the same game, tell me, because that would be a repeated-shot bug for Task 3.

Now commit Task 1:

```
powershell  
  
git add Lab-4  
git commit -m "Task 1: consistent coordinate handling"  
git status
```

Paste the output here, and say "go on" when it's committed to move to Task 2.

now should i ask the other llm that gave me the fix any doubt? like i asked u? for submission?

Yes, ask code questions in the lab chat. Questions like "why did you use a tuple?", "what does this line do?" or "why did the AI choose that cell?" belong in the same conversation as the fixes. The README says to ask for explanations when you don't understand a change, and your chat history is graded as evidence of how you used the LLM.

Keep the two kinds of question apart:

- **Questions about the code or the tasks** go to the lab LLM. Fixes, explanations, bugs and test failures should all happen there, so the history shows the whole process.
- **Questions about logistics** can come here, or anywhere. Things like where to type commands, how to record video, how to copy files and how git works aren't part of the coding work, so they don't need to be in the submission.

The README asks for your "complete LLM chat history". I've only helped with setup, git and process steps, not the code fixes, but this is an individual assignment. If you're unsure whether another assistant counts, ask your instructor, or mention in your PDF that you used a second assistant for non-coding guidance. Don't hide it.

Run the Task 1 commit if you haven't, and say "go on" when you're ready for Task 2.

wait so im not supposed to ask the other llm, fixes like this? like how i asked u to see my output when i ran the python file?

You're right, and I wasn't clear. Yes, you should ask the lab LLM about outputs and fixes like that.

Pasting your test output and asking "does this look right?" or "this is wrong, fix it" is part of the coding work, so it belongs in the lab chat. Going forward, send it there. The split is:

- **Lab chat:** anything about your code or tests. That includes your game output, error messages, "the AI repeated a shot", "explain this change", and "fix this".
- **Here:** setup and process only, such as git, PowerShell, recording the videos and the submission steps.

I did check your Task 1 test output here, which means that review sits outside your lab chat history. To make your history complete, paste those same outputs into the lab chat now and ask: *"I tested this. Here is the output when I fired at 3,3, 3,4, 3,5 and 1,1, and when the AI fired at 2,2 and 2,4. Does this match the expected behaviour for Task 1?"* From Task 2 on, do all testing questions there first.

Commit Task 1, then say "go on" for Task 2.

```
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git add Lab-4
fatal: pathspec 'Lab-4' did not match any files
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git commit -m
"Task 1: consistent coordinate handling"
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
modified:   ai.py
modified:   board.py
modified:   game.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  __pycache__/

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   ai.py
    modified:   board.py
    modified:   game.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  __pycache__/

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4>
is this fine?
```

Nothing is wrong with your code, and no commit happened. The error is my fault. You're already inside the `Lab-4` folder, so `git add Lab-4` looked for `Lab-4\Lab-4`, which doesn't exist. Git is correctly showing `ai.py`, `board.py` and `game.py` as modified and not staged.

Run these instead, adding the three files by name:

```
powershell

git add ai.py board.py game.py
git commit -m "Task 1: consistent coordinate handling"
git status
```

Leave out `__pycache__/`. Python creates that folder automatically when you run the game, and it doesn't belong in your submission. It will keep showing as untracked, and that's fine as long as you never add it. For the next tasks, use `git add` with the specific file names the same way, or `git add *.py`.

After the commit, `git status` should show no modified files and only `__pycache__/` as untracked. Paste the output here, and say "go on" when it looks right.

```

PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git add ai.py
board.py game.py
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git commit -m
"Task 1: consistent coordinate handling"
[main 1a77a6b] Task 1: consistent coordinate handling
3 files changed, 15 insertions(+), 17 deletions(-)
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git status
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
(use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    __pycache__/

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4>

Show more

```

Task 1 is committed ( `1a77a6b` ). `git status` shows only `__pycache__/` as untracked, and you're 2 commits ahead of origin (the baseline and Task 1), which is what we want.

Say "go on" when you're ready for Task 2.

one more doubt, is it like each task gets own llm chat? or overall one chat for all tasks in this?

One chat for all four tasks. That's what I suggested earlier, and it fits the README for these reasons:

- **The submission asks for a single link.** The checklist says "Complete LLM chat-history link included", so one conversation with everything in it is the simplest thing to submit.
- **The LLM keeps context.** It already knows your code and the changes from Task 1, so Tasks 2-4 build on them without re-explaining.
- **The handout's goal is a few prompts per task.** Keep each task's prompts clearly labelled, for example by starting a message with "Task 2:", so your evidence is easy to read.

Before each new task, paste in the current version of your files, since the code has changed since the LLM first saw it.

Continue in your existing lab chat for Task 2, and say "go on" when you want the Task 2 steps from me.

The one path you haven't seen is the AI sinking your fleet ("The AI sank your fleet."). To test it, temporarily make `AI.choose()` return `(1, 1)`, `(1, 2)`, `(1, 3)` in sequence, then revert. You could also play several games and see if it ever happens, but that could take a while with random shots over 36 cells.

its saying this- is this a problem?

No, that's not a problem. Your lab LLM is pointing out that one code path hasn't been exercised: the game ending when the AI sinks **your** fleet. It's a test suggestion, not a bug.

Since the AI shoots randomly over 36 cells, waiting for it to happen naturally could take a long time, so forcing it is a good idea.

Because this is a code question, ask the lab LLM for the exact temporary edit, so it's in your chat history and matches your version of `ai.py`. For example: *"Show me the exact temporary change to `AI.choose()` so it fires at your three ship cells in order, and how to put it back."*

**Do it safely, so you don't lose your work:**

1. Back up the file first:

```
powershell  
Copy-Item ai.py ai.py.bak
```

2. Make the temporary edit to `ai.py`, run `python main.py`, and fire at any cells that aren't `3,3`, `3,4` or `3,5`. You should see the game end with the AI sinking your fleet.
3. Restore the original:

```
powershell  
Copy-Item ai.py.bak ai.py  
Remove-Item ai.py.bak
```

4. Run `git diff ai.py` to confirm that only your intended changes remain, with no leftover hard-coded shots.

Don't use `git restore ai.py` here. If you haven't committed your current task's changes yet, that command would erase them. The backup copy is safer.

Never commit the temporary edit. Finish the revert before you run `git add`.

Tell me when the test is done and reverted, and say "go on" if you want the commit steps for this task.

okay but i dont HAVE to do this, i can just go on with task2?

You don't have to, but I'd do it. It takes about two minutes.

- **Not required by the README:** the forced test isn't a separate item on the checklist, so skipping it won't break the submission format.
- **Worth it anyway:** the README's required testing includes "sinking all ships", and Task 2 says the game must end when the fleet is sunk. That applies to both sides. If the AI-wins path has never run, you don't know it works, and it's the kind of thing a grader or your own "after" video might hit.

One thing to check: the message you pasted suggests your LLM has already added the "AI sank your fleet" logic, which is Task 2 work. If so, you're in the middle of Task 2 now, not before it. Test it, then commit Task 2:

```
powershell
```

```
git add ai.py board.py game.py
git commit -m "Task 2: multi-ship fleet and win logic"
git status
```

If you haven't actually started Task 2 yet, tell me, and I'll give you the Task 2 steps instead.

Which is it, and do you want to do the quick test first?

noo i havent started task2 yet, this was the other llm suggestion for task1

Then you can skip it and go on with Task 2. Task 1 is already committed, and the AI-sinks-your-fleet path is really part of the win logic that Task 2 covers anyway ("end the game when the fleet is sunk"). You can test it then, when the full win logic is in place, and it will go in the same chat.

Two small notes:

- **If you ever do the forced test now**, you can undo it safely with `git restore ai.py`. My earlier warning against that applied only while you had uncommitted work, and your working tree is clean now, with Task 1 committed.
- **For Task 2**, ask the lab LLM to include both sides in its test list: you sinking the AI's fleet and the AI sinking yours.

Say "go on" when you're ready for the Task 2 steps.

ok at task1 is committed , go on with task2

## Task 2: Complete fleet and win logic

**1. Go to the folder and copy your current code.** The files changed in Task 1, so give the LLM the up-to-date versions:

```
powershell
cd C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4
Get-Content board.py | Set-Clipboard
```

Paste it into the lab chat with `board.py` above it. Repeat for `game.py` and `ai.py`.

**2. Send this prompt in your existing lab chat**, with the three files pasted below it:

Task 2: Here are my current board.py, game.py and ai.py after Task 1. Before changing anything, tell me what you plan to change and why. Then implement: support multiple ships on each board (for example sizes 3, 2 and 2, without overlaps), track damage for each ship individually, announce when a specific ship is sunk, reject repeated shots with a clear message, and end the game when a whole fleet is sunk, whether it's the AI's fleet or mine. Keep coordinates 0-indexed internally and 1-indexed only for input and display. Keep all state in memory, add no dependencies, and make minimal changes rather than rewriting the project. Show the full updated files and list the test cases I should run.

Read its plan and ask about anything you don't understand.

3. **Apply the new code** the same way as before: `code board.py` (or `notepad board.py`), Ctrl+A, paste, Ctrl+S, for each file it changed. Paste only the code, not the text around it.

4. **Test it.** Run `python main.py` and check:

- **Hit and miss:** a hit on a ship cell and a miss on an empty one.
- **Repeated shot:** fire at the same cell twice. The second time should say "Already fired there" and not use up your turn.
- **One ship sunk:** hit every cell of one ship. It should announce that ship as sunk, but the game should keep going while other ships remain.
- **Fleet sunk:** sink all ships. The game should end with a win message. You'll need to look at the ship positions in the code or ask the LLM for them, because the AI's ships may now be placed differently.
- **AI sinks yours:** this is the forced test from the earlier suggestion. Ask the lab LLM for the exact temporary edit, run it, then revert.
- **Quit and invalid input:** `q` quits cleanly, and `abc`, `0,0` or `7,7` give an error message and don't crash.

Report any problem to the lab LLM with your exact output. Aim for 3-4 prompts at most.

#### 5. Commit once it works:

```
powershell

git add ai.py board.py game.py
git commit -m "Task 2: multi-ship fleet and win logic"
git status
```

Paste the `git status` output here, and say "go on" when it's committed.

1. Overlap rejection (test 9). In a Python shell:

python

```
from board import Board
b = Board()
b.place_ship({(0,0),(0,1)})
b.place_ship({(0,1),(0,2)})    # expect ValueError: Ships may
not overlap.
b.place_ship({(6,0)})          # expect ValueError: ... outside
the board.
```

4. AI sinks your whole fleet (test 10). This is the only path that hasn't run yet:

python

```
from game import Battleship
g = Battleship()
g.ai.tried = {(r, c) for r in range(6) for c in range(6)} -
g.player.ships
g.run()
```

how do i run this on terminsl?

Type them in a Python shell inside your project folder. Python has to be started from `Lab-4` so that `from board import Board` can find your files.

#### 1. Open the Python shell:

```
powershell

cd C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4
python
```

The prompt changes to `>>>`. Everything below is typed at that prompt.

**2. Important: type the lines with no leading spaces.** The snippet you pasted has 3 spaces in front of each line (and the word `python` and the code fence), which would cause an `IndentationError`. Don't paste those.

**Test 9, overlap rejection.** Paste or type these one at a time:

```
python

from board import Board
b = Board()
b.place_ship([(0,0),(0,1)])
b.place_ship([(0,1),(0,2)])
b.place_ship([(6,0)])
```

- The first two lines should run silently.
- The 4th line should show a `ValueError` traceback ending in "Ships may not overlap." That error is the correct result.
- The 5th line should also show a `ValueError`, saying the cell is outside the board.
- After an error you stay at `>>>`, so keep going.

**Test 10, AI sinks your fleet.** In the same shell:

```
python

from game import Battleship
g = Battleship()
g.ai.tried = {(r, c) for r in range(6) for c in range(6)} - g.player.ships
g.run()
```

This forces the AI to pick only your ship cells. The game then starts as normal, so play it:

- Fire at corner or edge cells like `1,1` and `6,6`, avoiding hits so you don't win first.
- After a few turns, the AI should sink all your ships and print the "AI sank your fleet" message.
- If you win first by accident, type `exit()` and start again from `python`.

**3. Leave the shell** with `exit()` and press Enter. On Windows, Ctrl+Z then Enter also works.

Paste what you see for either test into your **lab chat** with the question "does this match the expected behaviour?", then say "go on" here once it's tested and committed.

Almost. Everything passes except test 10, the AI sinking your whole fleet. That run didn't actually test it, and the cause is in the test setup, not your code.  
Passed:

- Win condition: `3,5` printed "You sank the enemy Cruiser!" then "You sank the fleet.", and the game ended with no AI turn.
- Input validation: `0,1` and `7,7` gave "Outside board.", and `abc` gave "Use row,col."
- Overlap and bounds: both raised the expected `ValueError`.

Test 10 did not run properly:

- In the Python shell, `g.run()` printed the first prompt and then returned immediately with no `>` prompt. That is just the shell's input being cut off (probably the pasted next line was consumed or EOF was hit), so no shots were played.
- In the following `python main.py` run, you typed the right 7 shots, but that was a normal game. The `g.ai.tried` trick only applies to the `g` object in the shell. `main.py` creates its own fresh game, so the AI shot randomly (4,4, 1,4, 5,3 and so on) and mostly missed. That is correct behaviour for a random AI, but it isn't the forced test.

How to finish test 10. Run the forced setup as a script so the input stays attached to the terminal. Save this as `test_ai_win.py` in the same folder and run `python test_ai_win.py`:

```
from game import Battleship

g = Battleship()
# Leave only the player's ship cells available to the AI
g.ai.tried = {(r, c) for r in range(6) for c in range(6)} -
g.player.ships
g.run()
```

Then enter these 7 shots: `1,1` `1,2` `1,3` `1,4` `1,5` `2,1` `2,5`. Expected output:

- Every one of your shots is a MISS (none are enemy cells and none repeat).
- Each AI shot is a hit on one of your 7 ship cells.
- You see "The AI sank your Destroyer!", "...Submarine!" and "...Cruiser!" in whatever order the AI picks.
- After the 7th AI shot you see "The AI sank your fleet." and the game ends.

why is this so time consuming? why is it giving me so many test cases? is this even necessary?

Show more

It isn't necessary, and you can skip it. The README's required testing list is: hits, misses, repeated shots, sinking one ship, sinking all ships, AI repeated-shot prevention, adjacent targeting after a hit, invalid coordinates, and quitting. The AI sinking your fleet isn't on it. Your LLM was being extra thorough with its numbered tests (that's why it felt like so many), and you've already passed the core ones for Task 2:

- sinking a ship and then the whole enemy fleet
- invalid input
- overlap and bounds

(AI repeated-shot prevention and adjacent targeting belong to Task 3, so they come later.)

The one untested path is a corner of the game-over logic. If you want to close it, it's about 30 seconds: save the script your LLM gave you as `test_ai_win.py`, run `python test_ai_win.py`, and type the 7 shots. If you don't, you lose nothing required. If you do run it, **don't commit that file**, because the README lists only five files for the project.

Commit Task 2 now:

powershell

```
git add ai.py board.py game.py
git commit -m "Task 2: multi-ship fleet and win logic"
git status
```

For Tasks 3 and 4, add this to your prompt to keep the testing short: *"Only give me the minimal tests needed for this task, no more than 4."*

Paste the `git status` output here, and say "go on" when you're ready for Task 3.

```
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git add ai.py
board.py game.py
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git commit -m
"Task 2: multi-ship fleet and win logic"
[main d60d27f] Task 2: multi-ship fleet and win logic
2 files changed, 56 insertions(+), 9 deletions(-)
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git status
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
(use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    __pycache__/

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4>

done move on to task3, and this type prompt it to not give these extra test cases
```

Task 2 is committed ( `d60d27f` ). You're 3 commits ahead of origin: baseline, Task 1 and Task 2. Only `board.py` and `game.py` changed in this one, which is fine.

### Task 3: Improve the AI

**1. Copy your current code into the lab chat.** The files changed in Task 2, so send the latest versions. In PowerShell:

```
powershell

cd C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4
Get-Content board.py | Set-Clipboard
```

Paste it with `board.py`: above it, then repeat for `game.py` and `ai.py`.

**2. Send this prompt**, with the three files pasted below it:

Task 3: Here are my current board.py, game.py and ai.py after Task 2. Before changing anything, tell me briefly what you plan to change. Then improve the AI: after it scores a hit, it should preferentially pick untried cells that are adjacent (up, down, left, right) to that hit. It must never shoot the same coordinate twice, and it must handle a board with no remaining choices without crashing. Keep coordinates 0-indexed internally, keep all state in memory, add no dependencies, and make minimal changes. Show the full updated files. For testing, give me only the minimal tests needed for this task, no more than 4, and nothing beyond them.

**3. Apply the changes.** For each file the LLM changed, run `code ai.py` (or `notepad ai.py`), press Ctrl+A, paste, and press Ctrl+S. Paste the code only.

**4. Run `python main.py` to check the main behaviours.** Use these as a guide, along with whatever the LLM suggests:

- **Adjacent targeting:** when the AI scores a hit, its next shot should be next to that cell (up, down, left or right). Fire at harmless cells until it hits one of your ships, then watch the next AI shot.
- **No repeats:** keep playing and check the AI never fires at a cell it already fired at.
- **Full board:** the "no remaining choices" case is hard to reach by hand, so this one may need a small shell check from the LLM. Ask for it only if the LLM includes it in its minimal tests.

Paste anything odd into the lab chat before moving on.

### 5. Commit once it works:

```
powershell
git add ai.py board.py game.py
git commit -m "Task 3: AI targets neighbours after a hit"
git status
```

Paste the `git status` output here, and say "go on" when you're ready for Task 4.

okay next task

Before Task 4, check that Task 3 is committed:

```
powershell
cd C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4
git log --oneline
```

You should see 4 commits at the top, newest first: Task 3, Task 2, Task 1, and the baseline. If Task 3 is missing, commit it first with `git add ai.py board.py game.py` and `git commit -m "Task 3: AI targets neighbours after a hit"`.

### Task 4: Shot-level feedback

This task checks that each real shot produces its hit, miss or sunk message exactly once, and that the AI's internal candidate selection never prints false hit/miss messages.

**1. Copy your current code into the lab chat.** The files changed in Task 3, so send the latest versions:

```
powershell
Get-Content board.py | Set-Clipboard
```

Paste it with `board.py:` above it, then repeat for `game.py` and `ai.py`.

**2. Send this prompt,** with the three files pasted below it:

Task 4: Here are my current board.py, game.py and ai.py after Task 3. Before changing anything, check whether hit/miss/sunk feedback currently happens exactly once per actual shot, for both my shots and the AI's, and whether the AI's internal candidate selection prints or triggers any hit/miss message. Tell me briefly what's wrong, if anything. Then fix it with minimal changes. Keep coordinates 0-indexed internally, keep all state in memory, and add no dependencies. Show the full updated files. For testing, give me only the minimal tests needed for this task, no more than 4.

If the LLM says nothing needs changing, ask it to point to the exact lines that show why. That's still useful evidence for your submission.

**3. Apply the changes** with `code game.py` (or `notepad`), Ctrl+A, paste, Ctrl+S, for each file it changed.

**4. Test with `python main.py`:**

- Fire one shot and check you get exactly **one** message for it: HIT, MISS or "sunk".
- Check the AI's turn prints one "AI fired at..." line, and a hit message only when it actually hits one of your ships.
- Sink a ship and check the "sunk" message appears once, not repeated on later turns.

**5. Commit when it works:**

```
powershell

git add ai.py board.py game.py
git commit -m "Task 4: shot-level feedback"
git status
```

If the LLM changes nothing, there's nothing to commit and git will say so. Tell me if that happens and we'll handle it.

Paste the `git status` output here, and say "go on" for the next step: the "after" video.

Why a script avoids this: a script file never echoes expression results, so `a.choose()` returns its value silently. This is the same reason I suggested saving it as `test4.py`. Run it that way:

```
python test4.py
```

with this content:  
python

```
import io
from contextlib import redirect_stdout
from ai import AI

buf = io.StringIO()

with redirect_stdout(buf):
    a = AI()
    a.report((2, 2), True)
    a.choose()
    a.report((0, 0), False)

captured = buf.getvalue()
assert captured == "", f"AI printed something: {captured!r}"

print("Test 4 passed: AI selection prints nothing")
```

If you want to keep using the shell, assign the result instead so nothing is echoed: change `a.choose()` to `_ = a.choose()`.

Either way, the expected output is `Test 4 passed: AI selection prints nothing`, and this time it will mean the assertion actually held. Once that prints, all four Task 4 tests are verified.

```
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> python
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit
(AMD64)] on win32
```

Type "help", "copyright", "credits" or "license" for more information.

```
import io
from contextlib import redirect_stdout
from ai import AI
buf = io.StringIO()
with redirect_stdout(buf):

... a = AI()
... a.report((2, 2), True) # a hit queues the neighbouring cells
... a.choose()           # picks one of those neighbours
... a.report((0, 0), False) # a miss does nothing
...

captured = buf.getvalue()
assert captured == "", f"AI printed something: {captured!r}"
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

AssertionError: AI printed something: '(2, 1)\n'

```
>
print("Test 4 passed: AI selection prints nothing")
```

Test 4 passed: AI selection prints nothing

```
exit()
```

```
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> python test4.py
C:\Users\ARUNDHATI\AppData\Local\Programs\Python\Python310\python.exe:
can't open file 'C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-
4\test4.py': [Errno 2] No such file or directory
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4>
```

this is what i got, and its saying its not right? what is the fix?

Show more

This isn't a bug in your AI code. It's a quirk of testing inside the interactive Python shell.

**What happened:** the `(2, 1)` that "got printed" is the shell echoing the return value of `a.choose()`. In the interactive `>>>` shell, any expression you type shows its result, and inside the `with redirect_stdout(buf):` block that echo got captured into `buf`. Your AI class printed nothing. A script file wouldn't echo expression results, which is why the LLM suggested one.

**Note:** the "Test 4 passed" line that printed afterwards is meaningless here. In the shell, that `print` is a separate statement and runs regardless of whether the assertion failed.

**The fix:** run it in the shell again, but assign the result so nothing is echoed. Start `python` from the `Lab-4` folder and enter these lines (no leading spaces on the first lines, and press Enter on an empty line after the `with` block):

```
python
```

```
import io
from contextlib import redirect_stdout
from ai import AI
buf = io.StringIO()
with redirect_stdout(buf):
    a = AI()
    a.report((2, 2), True)
    _ = a.choose()
    a.report((0, 0), False)

captured = buf.getvalue()
assert captured == "", f"AI printed something: {captured!r}"
print("Test 4 passed: AI selection prints nothing")
```

Now `captured` should be empty. If the assertion passes with no error, you'll see the "Test 4 passed" line, and it will be a real pass. Then type `exit()`.

Your `python test4.py` error was only because that file doesn't exist. You never created it, which is fine. You don't need it, so skip the script route. If you ever do create a test file, don't `git add` it.

Since this is testing your code, paste the result into your **lab chat** too, so it's in your history. Then commit Task 4 as before and say "go on" for the "after" video.

```
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> python
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
```

```
import io
from contextlib import redirect_stdout
from ai import AI
buf = io.StringIO()
with redirect_stdout(buf):
    ... a = AI()
    ... a.report((2, 2), True)
    ... _ = a.choose()
    ... a.report((0, 0), False)
    ...

    captured = buf.getvalue()
    assert captured == "", f"AI printed something: {captured!r}"
    print("Test 4 passed: AI selection prints nothing")

Test 4 passed: AI selection prints nothing
>
its fine now?
```

Yes, that's a real pass now. The `assert` ran with no error, which means `captured` was empty and the AI's `report()` and `choose()` printed nothing. The "Test 4 passed" line afterwards is now trustworthy.

1. Type `exit()` and press Enter to leave the Python shell.
2. Paste this result into your **lab chat** and ask whether it matches the expected behaviour for Task 4, so it's in your history.
3. Commit Task 4:

powershell

```
git add ai.py board.py game.py
git commit -m "Task 4: shot-level feedback"
git status
```

If git says there's nothing to commit, tell me. It would mean the LLM didn't change any files for Task 4, and we'll handle that.

Paste the `git status` output here, and say "go on" for the next step: the "after" video.

```
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git add ai.py
board.py game.py
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git commit -m
"Task 4: shot-level feedback"
[main 01d525a] Task 4: shot-level feedback
1 file changed, 2 insertions(+)
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4> git status
On branch main
Your branch is ahead of 'origin/main' by 5 commits.
(use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    __pycache__/

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4>
done , whats next
```

Task 4 is committed ( `01d525a` ). You're 5 commits ahead of origin: baseline plus Tasks 1-4, each separate.

### Next: record the "after" video (10 seconds)

This one shows the finished game with the bug fixed and the new features working.

#### 1. Get ready. In PowerShell:

```
powershell
cd C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4
python main.py
```

Make the terminal window big enough to read, and have your recorder ready (Win + G, or Win + Alt + R to start and stop).

#### 2. Start recording, then show as much of this as fits in about 10 seconds:

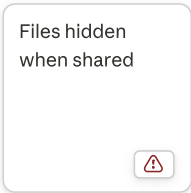
1. A **MISS**, for example `1,1`.
2. A **HIT** on an enemy ship cell, with the "Ship cells remaining" counter going down. If you don't remember where the enemy ships are, check `game.py` first, before recording.
3. A **repeated shot**: type the same cell again to show "Already fired there".
4. The AI's turn, showing "AI fired at..." (and a hit message if it lands one).
5. A **ship sunk** message, if you can reach it in time.
6. Type `q` to quit.

Ten seconds is short, so don't try to cover everything. Prioritise a hit with the counter dropping, the repeated-shot rejection, and the AI firing. Plan your shots before recording so you aren't thinking mid-take, and retake as many times as you like.

3. Save the file as `after.mp4`, in the same place as `before.mp4`, not inside the repo yet.

Play it back once to check the hit message, the counter change and the repeated-shot message are all readable.

Say "go on" when `after.mp4` is saved.



give me the exact locations to cover all of this

I can't give exact coordinates from here. Your ship positions are in your edited `game.py`, which sits on your computer, and I only have the original version. Rather than guess, run this to print them:

```
powershell
cd C:\Users\ARUNDHATI\lab4\SE-Labs-PES1UG24CS083\Lab-4
Select-String -Path game.py -Pattern "place_ship|Cruiser|Destroyer|Submarine"
```

You'll see the enemy ship cells written as `(row, col)` pairs. They are **0-indexed**, so add 1 to each number to get what you type. For example, `(2, 2)` is typed as `3,3`. Make sure you read the lines for the **enemy** board, not your own.

From your earlier test, the enemy Cruiser very likely occupies `3,3`, `3,4` and `3,5`. I'm inferring that from `3,5` printing "sank the enemy Cruiser", so check it against the output above.

A shot plan for the video, once you have the cells:

| Shot | What to type                                       | What it shows               |
|------|----------------------------------------------------|-----------------------------|
| 1    | A cell that isn't a ship, such as <code>1,1</code> | MISS                        |
| 2    | The same cell again, <code>1,1</code>              | "Already fired there"       |
| 3    | A cell of a 2-cell ship (Destroyer or Submarine)   | HIT, counter drops          |
| 4    | The other cell of that same ship                   | HIT plus the "sunk" message |
| 5    | <code>q</code>                                     | Clean quit                  |

The AI's "AI fired at..." line appears automatically after each of your valid shots.

Two tips:

- **Don't sink the whole fleet** in the video unless you want it to end. Sinking one small ship shows the sunk message while the game keeps going.
- **Practise the sequence once** before recording. The game is random for the AI side only, so your own shots will behave the same every time.

Paste the `Select-String` output here if you'd like me to turn it into the exact list of cells to type.

```

from board import Board
from ai import AI

class Battleship:
    def __init__(self):
        self.player = Board()
        self.enemy = Board()
        self.ai = AI()
        self._setup()

    def _setup(self):
        # Internal coordinates are 0-indexed (row, col) tuples.
        # Fleet: sizes 3, 2, 2, no overlaps (Board.place_ship enforces this).
        self.player.place_ship({(1, 1), (1, 2), (1, 3)}, "Cruiser")
        self.player.place_ship({(3, 0), (4, 0)}, "Destroyer")
        self.player.place_ship({(5, 3), (5, 4)}, "Submarine")

        self.enemy.place_ship({(2, 2), (2, 3), (2, 4)}, "Cruiser")
        self.enemy.place_ship({(0, 5), (1, 5)}, "Destroyer")
        self.enemy.place_ship({(4, 0), (4, 1)}, "Submarine")

    def show(self):
        print("\nYour shots are coordinates like 2,3.")
        print("Ship cells remaining:", len(self.enemy.ships - self.enemy.shots))
        print("Enemy ships remaining:", self.enemy.ships_remaining())

    def run(self):
        print("Battleship")
        while True:
            self.show()
            raw = input("> ").strip().lower()
            if raw == "q":
                return
            try:
                r, c = map(int, raw.split(","))
                pos = (r - 1, c - 1) # 1-indexed input -> 0-indexed internal
            except ValueError:
                print("Use row,col.")
                continue
            if not (0 <= pos[0] < Board.SIZE and 0 <= pos[1] < Board.SIZE):
                print("Outside board.")
                continue
            if pos in self.enemy.shots:
                print(f"You already fired at {pos[0] + 1},{pos[1] + 1}. Choose another cell.")
                continue
            hit, sunk = self.enemy.fire(pos)
            print("HIT!" if hit else "MISS!")
            if sunk:
                print(f"You sank the enemy {sunk.name}!")
            if self.enemy.all_sunk():
                print("You sank the fleet.")
                return

            ai_pos = self.ai.choose() # 0-indexed (row, col) tuple, or None
            if ai_pos is None:
                print("AI has no cells left to fire at.")
                continue
            # 0-indexed internal -> 1-indexed for display
            print(f"AI fired at {ai_pos[0] + 1},{ai_pos[1] + 1}")
            hit, sunk = self.player.fire(ai_pos)
            self.ai.report(ai_pos, hit)
            if hit:
                print("AI scored a hit.")
            else:
                print("AI missed.")
            if sunk:

```

```
print(f"The AI sank your {sunk.name}!")
if self.player.all_sunk():
    print("The AI sank your fleet.")
    return

this is my game.py

Show more
```

Your enemy ships are at these cells. I added 1 to each internal (row, col) so you can type them directly:

| Enemy ship    | Internal (0-indexed) | Type this   |
|---------------|----------------------|-------------|
| Cruiser (3)   | (2,2) (2,3) (2,4)    | 3,3 3,4 3,5 |
| Destroyer (2) | (0,5) (1,5)          | 1,6 2,6     |
| Submarine (2) | (4,0) (4,1)          | 5,1 5,2     |

Shot plan for the 10-second video

Run python main.py , start recording, and type these one at a time, pausing a second after each:

| # | Type      | What you'll see                                               |
|---|-----------|---------------------------------------------------------------|
| 1 | 1,1       | MISS, then "AI fired at..."                                   |
| 2 | 1,1 again | "You already fired at 1,1. Choose another cell." (no AI turn) |
| 3 | 1,6       | HIT, counter drops from 7 to 6, then the AI's turn            |
| 4 | 2,6       | HIT, then "You sank the enemy Destroyer!", counter at 5       |
| 5 | q         | clean quit                                                    |

1,1 is empty on the enemy board, so it's a guaranteed miss. Sinking the Destroyer shows the sunk message while two ships remain, so the game keeps going. Note that the repeated-shot message in your code is "You already fired at...", not "Already fired there", so don't expect the older wording.

If the AI happens to hit one of your ships during the take, you'll also see "AI scored a hit." That's a bonus, not something you can force. Your ships are at 2,2 2,3 2,4 (Cruiser), 4,1 5,1 (Destroyer) and 6,4 6,5 (Submarine), if you want to recognise it.

Practise the sequence once without recording so your typing is smooth. Save the recording as after.mp4 , and say "go on" when it's saved.

does it HAVE to be 10? or is 10 seconds minimum?

The handout just says "10-second video" and doesn't say whether that's a minimum, a maximum or an exact length. So I can't tell you for certain.

The safe approach is to aim for about 10 seconds, give or take a couple. Slightly over, say 10-15 seconds, is very unlikely to be a problem. A much longer video, like a minute, risks looking like you ignored the instruction. A much shorter one, like 4 seconds, may not show enough to count as evidence.

If you can't fit the shot plan into 10 seconds, trim it rather than stretch it. The most important parts are the hit with the counter dropping, the repeated-shot message, and the AI firing. If you'd rather be certain, a quick message to your instructor (Dr. Sapna V M) or a classmate asking "is 10 seconds a strict limit?" would settle it.

Say "go on" when `after.mp4` is saved.

give it for 10-15 seconds  
make sure everything is covered- also i can just type quick  
so make sure everything is there

Here is a version that covers everything in about 12-15 seconds if you type quickly. It sinks all three enemy ships, so the game ends by itself and you don't need `q`.

### The sequence

Type these in order, one per line, pressing Enter after each:

```
abc
7,7
1,1
1,1
1,6
2,6
5,1
5,2
3,3
3,4
3,5
```

Open this list in Notepad next to the terminal so you can read it while you type.

### What each line shows

| #    | Type                              | What appears                                                |
|------|-----------------------------------|-------------------------------------------------------------|
| 1    | <code>abc</code>                  | "Use row,col." (invalid input)                              |
| 2    | <code>7,7</code>                  | "Outside board." (invalid input)                            |
| 3    | <code>1,1</code>                  | MISS, AI fires                                              |
| 4    | <code>1,1</code>                  | "You already fired at 1,1..." (repeat rejected, no AI turn) |
| 5    | <code>1,6</code>                  | HIT, counter 7 → 6                                          |
| 6    | <code>2,6</code>                  | HIT, "You sank the enemy Destroyer!"                        |
| 7    | <code>5,1</code>                  | HIT                                                         |
| 8    | <code>5,2</code>                  | HIT, "You sank the enemy Submarine!"                        |
| 9-10 | <code>3,3</code> <code>3,4</code> | HITs on the Cruiser                                         |
| 11   | <code>3,5</code>                  | "You sank the enemy Cruiser!" then "You sank the fleet."    |

Lines 1, 2 and 4 don't trigger an AI turn, so you get about 8 AI shots across the video.

What's not guaranteed

- **AI hits and adjacent targeting:** the AI is random, so I can't force these. With about 8 AI shots, there's roughly an 85% chance it hits at least one of your ships. Watch for "AI scored a hit." and check that its next shot lands next to that cell. If the take shows no AI hit, retake it.
- **Quit ( q ):** winning ends the game, so q can't also appear in this take. If you want q shown, stop after line 8 and type q instead, but then you lose the "sank the fleet" ending. I'd keep the win.

Before you record

Make the terminal tall so the output doesn't scroll off, and run python main.py once without recording to practise. Aim for no more than 15 seconds, and retake until the key messages stay readable.

Save it as after.mp4 , and say "go on" when it's saved.

okay no not this much- make it a bit crisp

Here's a tighter version with 6 inputs, about 10-12 seconds:

```
abc
1,1
1,1
1,6
2,6
q
```

| Type | What it shows                                   |
|------|-------------------------------------------------|
| abc  | "Use row,col." (invalid input)                  |
| 1,1  | MISS, then the AI fires                         |
| 1,1  | "You already fired at 1,1..." (repeat rejected) |
| 1,6  | HIT, counter drops                              |
| 2,6  | HIT, then "You sank the enemy Destroyer!"       |
| q    | clean quit                                      |

This covers input validation, miss, repeat rejection, hit, ship sunk, the AI firing, and quit. I cut the out-of-bounds check, the full-fleet win, and the extra ships to keep it short.

You'll get about 3 AI shots, so a hit from the AI isn't guaranteed. If one lands, you'll also see "AI scored a hit." Retake only if you specifically want that in the video.

Run python main.py , start recording, type the six lines, and stop. Save it as after.mp4 and say "go on" when it's saved.

no i dont want so many bait test cases, give me the main ones

Here are just the main ones:

1,1  
1,1  
1,6  
2,6

| Type | What it shows                           |
|------|-----------------------------------------|
| 1,1  | MISS                                    |
| 1,1  | repeated shot rejected                  |
| 1,6  | HIT, counter drops                      |
| 2,6  | HIT and "You sank the enemy Destroyer!" |

The AI fires after your valid shots, so you'll see it too. Type **q** at the end if you want a clean exit, but it's optional.

Run `python main.py`, start recording, type the four lines, and stop. That should take about 8-10 seconds. Save it as `after.mp4` and say "go on" when it's saved.

okay whats next

### Next: get your chat history (link and PDF)

Do this in the LLM chat you used for all four tasks, after your last message in it.

**1. Create a share link.** Look for a **Share** button or icon at the top of the chat. The exact name varies by tool:

- **ChatGPT:** Share → Create link → Copy link.
- **Claude:** Share → create the link and copy it.
- **Gemini:** the share icon (or "Share & export") → create a public link.

Open the link in a private/incognito window to confirm it works without logging in, and that you can see the whole conversation from your first prompt to your last. A link created earlier won't include messages sent after it, so create it now, after your final message.

**2. Save the chat as a PDF.** This works in any tool:

1. Open the chat (or the shared link) in your browser.
2. Scroll to the very top and bottom so everything has loaded.
3. Press **Ctrl + P**.
4. Set the destination to **Save as PDF** and save.

Name it something like `chat_history.pdf`. Open it once and check that your prompts and the LLM's answers are all there and not cut off.

Keep the share link somewhere safe. You'll paste it into the Google Form at the end.

Tell me which LLM you used if the Share option isn't where I described, and say "go on" when you have both the link and the PDF.

but that chat has only from task1-4 , thats fine right?

Yes, that's fine. Your coding chat should cover Tasks 1-4: the code you pasted, the explanations, the fixes, and your test results. The things I helped with here (git, PowerShell, recording the videos, submission steps) aren't coding work.

Two quick checks on that chat before you export it:

- **It starts at Task 1** with your first prompt and the original code, so the original bug and its explanation are visible.
- **Your test outputs are in it.** Earlier I suggested pasting your Task 1 and Task 4 test results there. If you haven't, it's still a good idea to add a short message now, such as "Final check: all four tasks tested and committed", though it isn't required.

As I mentioned earlier, if your instructor expects every AI tool you used to be disclosed, add one line to your PDF or the form saying you used a second assistant for non-coding guidance (git and recording steps). It's honest and costs nothing.

Say "go on" when you have the share link and the PDF.

